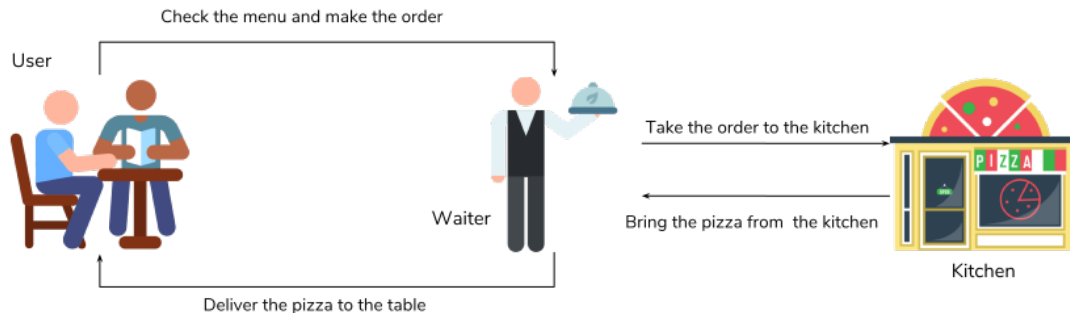
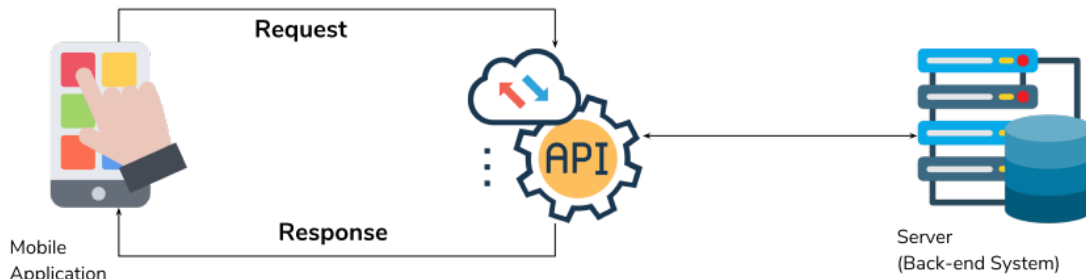


What is an API?

API (Application Program Interface) that allows two applications to talk to each other.

Example:



Technically:

API is Collection of **Pre-Written (Codes)** like- Packages, Classes and Interfaces with respective methods, fields and constructor.

Show the developer option and api call in fb or gmail - in Real time

API Allows users to use Pre-Written codes to manipulate their scripts.

Example- JDK- having 3 Basic components- Java Compiler, JVM and Java API

In java -what we are doing is just creating objects of class and calling the methods and performing the actions.

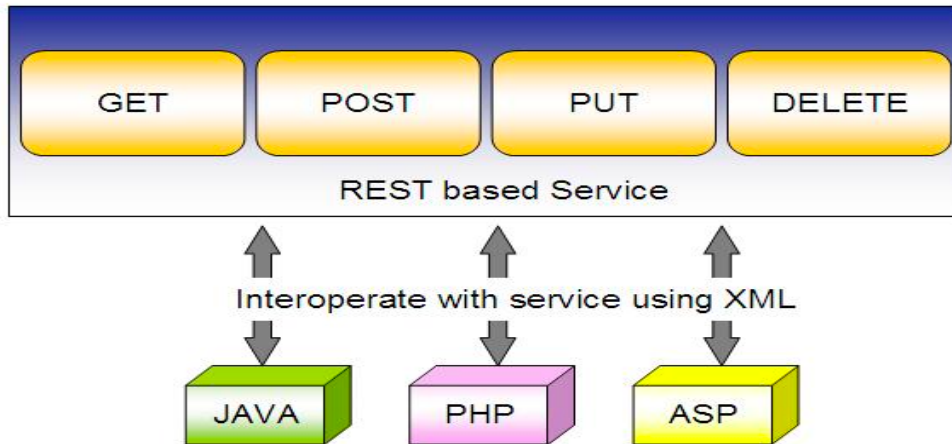
Selenium and RestAssured(Java API)- This also Pre_written Library or API we can say that. So these API -we are just using for further manipulation- we don't need to implement them.

If this API will be Uploaded into some server/Cloud i.e. API available on Web then We call this WebServices---that's all

So we can say that- All WebServices are API but all API are not WebServices.
Because all API will not be available over the web---very simple it is.

API's are used to connect the two different platforms. And JSON and XML are the schemas which are used to return the output from the REST and SOAP.

As opposed to **SOAP**, **REST** is not a protocol but an architectural style. ... It allows different messaging formats, such as HTML, JSON, XML, and plain text, while **SOAP** only allows XML. **REST** is also a more lightweight architecture, so **RESTful** web services have a better performance



The **key features of REST API** are as follows:

Represent everything with a unique ID; a URI

Stateless communication

Rest API-Make use of standard HTTP methods such as GET, POST, DELETE, and PUT

GET- The GET method is used to extract information from the given server using a given URI. While using GET request, it should only extract data and should have no other effect on the data. No Payload/Body required

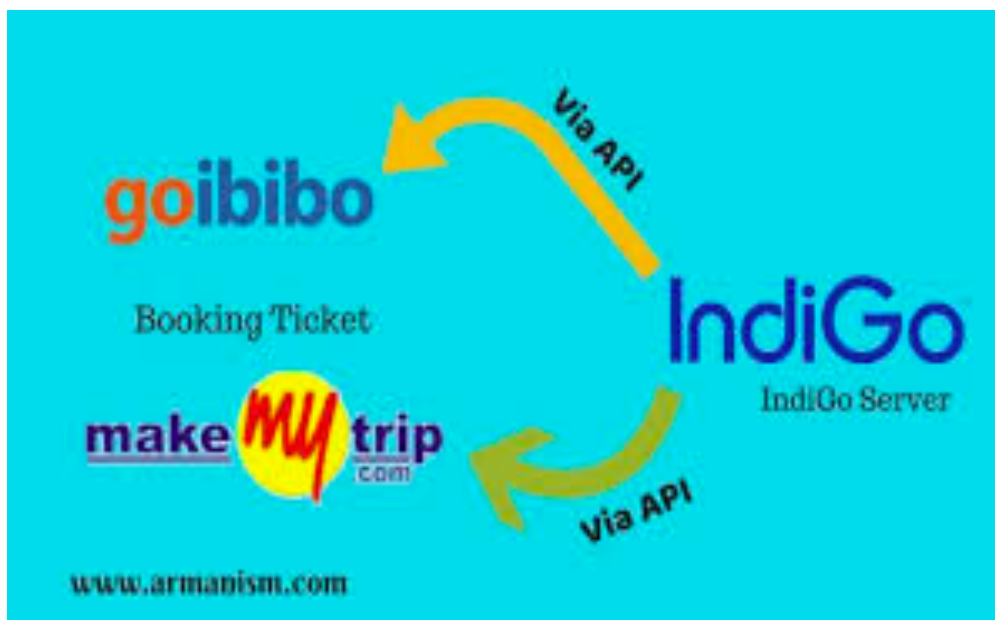
POST- A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.

PUT- Replaces all current representations of the target resource with the uploaded content.

DELETE- Removes all current representations of the target resource given by a URI.

Resources can have multiple representations

Basic Example: (No company will give you Direct Access to their DB- So we will use API)



Explain Basic goibibo Website all API calls

Booking Website

Google Map- used by no of applications like: zomato,swiggy, almost all the food delivery and many more.

Google exposes their map RestAPI- will see with an example.

Whether update

Traffic update?

(Client) Frontend ---->Hit RestAPI Req.---->Server(Third party)

Validate the contents and response back

(Client) Frontend <----Receive RestAPI Response.<----Server(Third party)

Sample Whether API example:

How to Develop API?

For weather update- some whether.com third party websites have a design that if you pass the city name will give you the temperature of that city--very simple example.

<http://restapi.demoga.com/utilities/weatherfull/city/hyderabad>

How does it work Internally?

Google Map-- in Firefox and Chrome..manually will see?

<https://developers.google.com/maps/documentation/urls/guide>

Explanation:

Google map Search API:

<https://www.google.com/maps/search/?api=1¶meters>

Try to understand this URL:

<https://www.google.com/maps/search/?api=1¶meters>

URI/Service/Parameters

We Will see the basic goibibo website to understand end points?

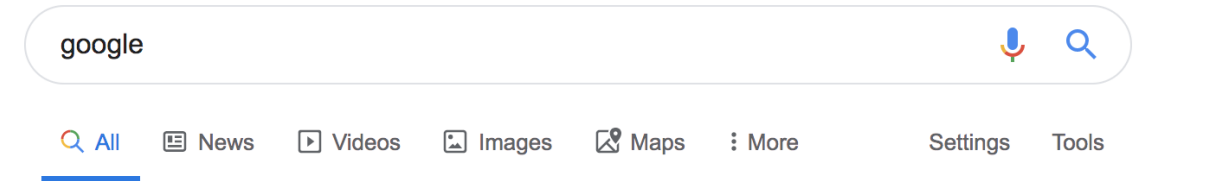
<https://www.goibibo.com/>- This is Hosting Website (Host/URI)

<https://www.goibibo.com/hotels/>- Hotel is one of the services

<https://www.goibibo.com/bus/#home>- bus is another service like that

End Point:

The place that APIs send requests and where the resource lives, is called an endpoint. Here- <https://www.google.com> is the end point where so many resources live like map,news,video,image etc.



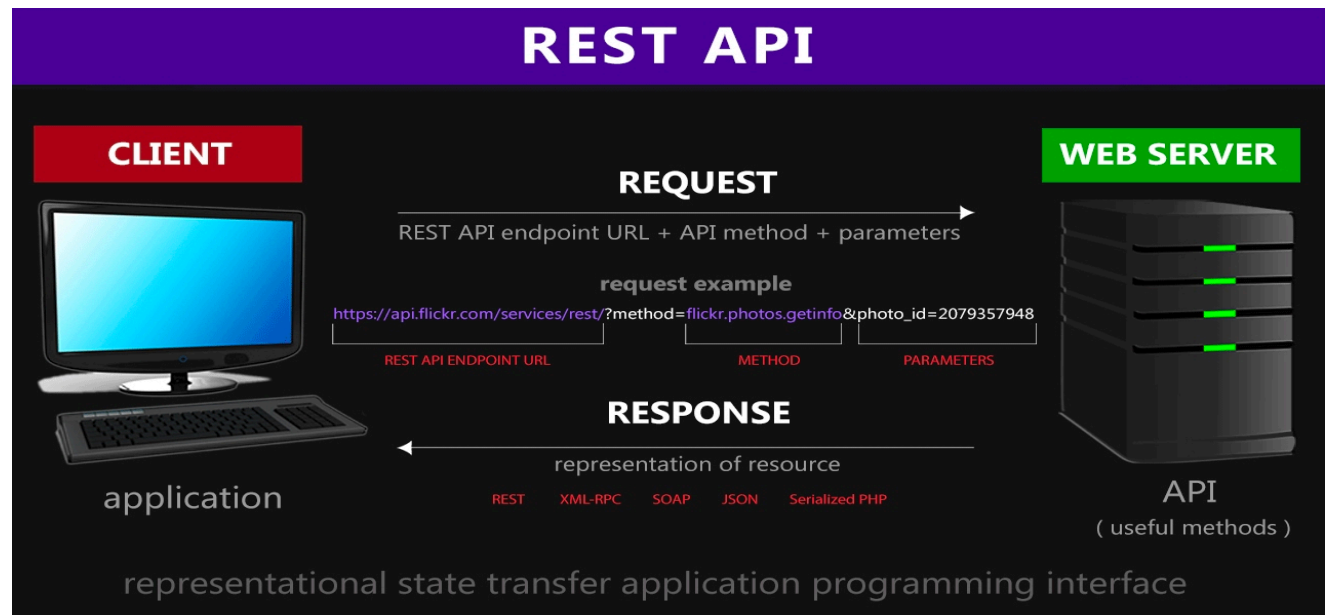
URI/HOST - <https://www.google.com>

Resource/Services - </maps/search/> (google offers multiple services map is one of the services)

Parameters - [?api=1¶meters](#)

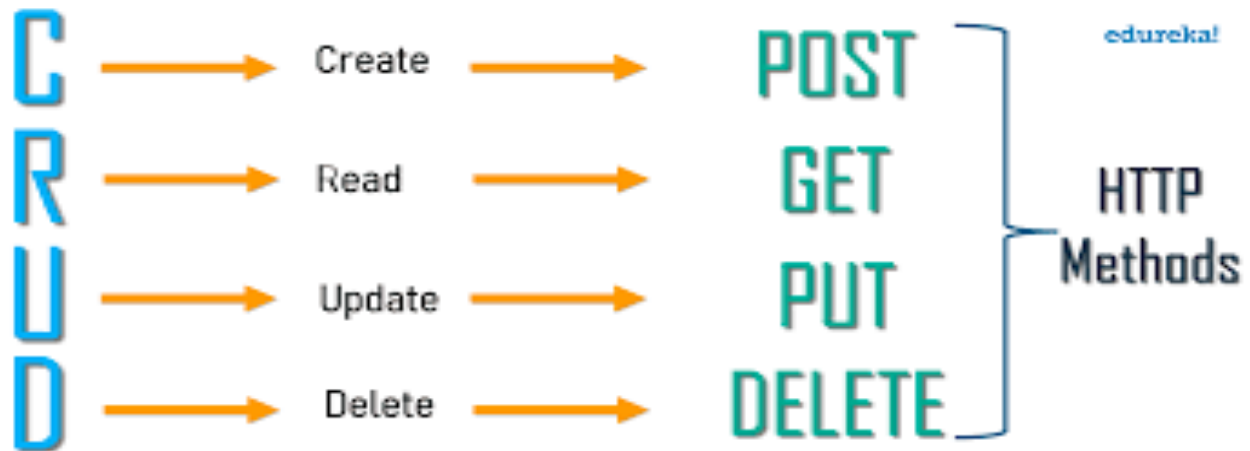
?(Question marks separate the Resources and Parameter)

Both URL and URI is protocol to retrieve the resource. URI stands for uniform resource identifier and URL stands for uniform resource locator. ... You see, a URI can be a name, locator, or both for an online resource where a URL is just the locator. URLs are a subset of URIs.



What are the types of http RestAPI Request we used mostly?

There are n no methods are there but most widely used methods are Get,Post,Put,Delete these all 4 methods called as CRUD- Operation.



GET- The GET method is used to extract information from the given server using a given URI. While using GET request, it should only extract data and should have no other effect on the data. No Payload/Body required.

In case of get request -we need to pass parameters as part of **URI/URL** to retrieve the Data.

POST- A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.

In case of Post request -we need to pass **Body/Payload** as xml/json to add/manipulate the Data.

Body/Payload:

PUT- Replaces all current representations of the target resource with the uploaded content.

In case of Put request -we need to pass **Body/Payload** as xml/json to Edit/manipulate the Data.

DELETE- Removes all current representations of the target resource given by a URI.

Resources can have multiple representations

In case of Delete request -we need to pass **Body/Payload** as xml/json to Delete/manipulate the Data.

Manual Testing Using PostMan- What is PostMan ? What are the different things are there?

--Install PostMan and Explanation

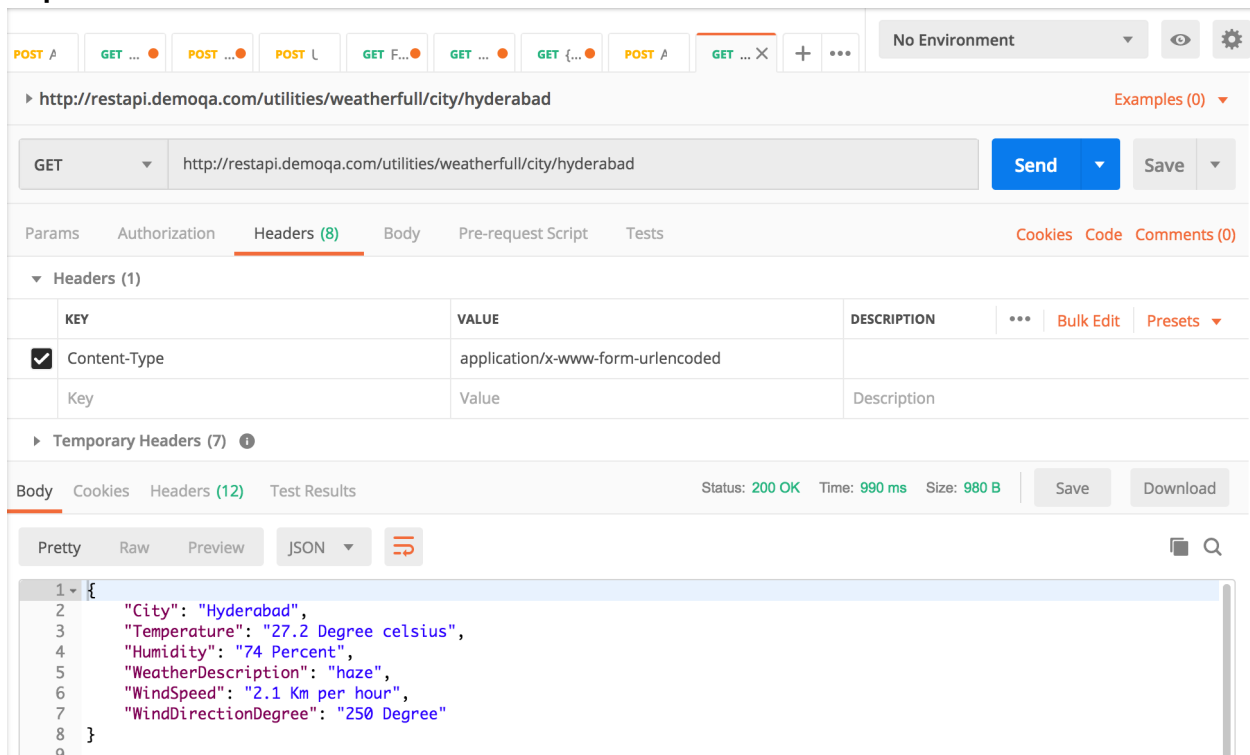
-Other tools also there but postMan is very easy to use

<https://chrome.google.com/webstore/detail/postman/fhbjgbiflinjbdgghehcdcdcbncdddomop//%40>

PostMan For Chrome Add on Extension or You can download as well.--Do Signup free

Understand basic terms then will see google map API?

Explanation of Postman?



Understanding different terms like Header, Body/Payload, cookies etc?

Header- to Authenticate the user, pass different input like json/xml/text etc.

Different Link to Perform API Testing & Practice:

- <https://reqres.in/>
- <https://httpbin.org/>
- <http://dummy.restapiexample.com/>
- <https://jsonplaceholder.typicode.com/>
- <https://fakerestapi.azurewebsites.net/>
- <https://www.programmableweb.com/apis/directory>
- <https://developers.google.com/maps/documentation>
- <https://developer.github.com/v3/repos/>

Understanding Google map API:

Google Search API?

Parameter we can pass:

1. By Place Name

<https://www.google.com/maps/search/?api=1&query=centurylink+field>

2. By using latitude/longitude coordinates as well as the place ID

<https://www.google.com/maps/search/?api=1&query=47.5951518,-122.3316393>

3. By using latitude/longitude etc.

https://www.google.com/maps/search/?api=1&query=47.5951518,-122.3316393&query_place_id=ChIJKxjxuaNqkFQR3CK6O1HNNqY

Using PostMan-Manual Testing Client for REST API:

<https://developers.google.com/maps/documentation/javascript/tutorial>

Get API Keys

How to generate api key for google API?

Follow the Instruction and create Google API key.

Go to >Identity Aware proxy>>Configure consent Screen>>Go to Credentials>>Select API Key

Copy the Key and keep for future use:

AIzaSyCLuzFNxbXlbSbXCDeRbZKUGjem0FpfQEs

Go to API Library>>Click on Place API>>Enable it>>Go to Secure connection> get the API key

AIzaSyCLuzFNxbXlbSbXCDeRbZKUGjem0FpfQEs (This will be your same api Keys)

Save it now.

Go to Maps community support links>>Select Place API>Go to Documentation

Click on Place search -we will get all the details

Now will go and Use- Search nearby Location API.

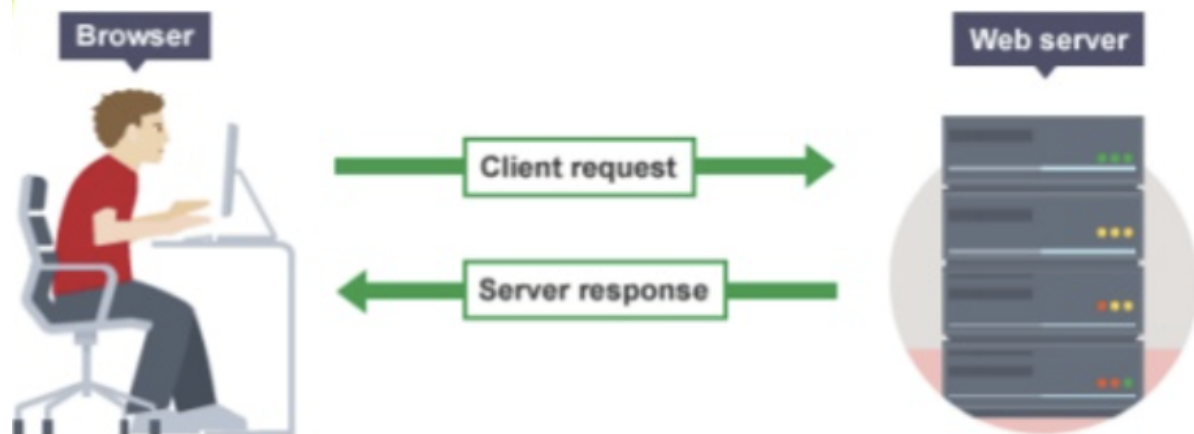
Lets See PostMan Tools First then we will go for the Search API.

SearchNearByLocation–See the preRequisite>>then Pass the Param

Use get API Example.

What is HTTP status code?

Whenever client requests to the Server, Server sends Pre defined (status/Error) **code** to the client. Status code defines what happened with the API call.



Client-Server Chat Application

All HTTP response status codes are separated into five classes or categories.

Status/Error Codes Series:

Series beginning with	What these codes indicate
1xx	These are informational messages to the web client, to inform the client that the server has received the request and is processing it. (You would normally not need to provide any response to these errors, as they are not actually errors.)
2xx	These codes indicate the success of the request sent from the client. (Again, in most cases, you would not need to provide any response for these HTTP Error Codes.)
3xx	These indicate some sort of redirection.
4xx	There is a problem with the request sent by the client.
5xx	The request was valid, but the server was unable to process the request. (The actual HTTP error code would indicate the reason for the failure.)

The first digit of the **status code** defines the class of **response**, while the last two digits do not have any classifying or categorization role But gives you actual code when merged with the first digit.

ALL Actual Response Codes:

1XX Informational		4XX Client Error Continued	
100	Continue	409	Conflict
101	Switching Protocols	410	Gone
102	Processing	411	Length Required
2XX Success		412	Precondition Failed
200	OK	413	Payload Too Large
201	Created	414	Request-URI Too Long
202	Accepted	415	Unsupported Media Type
203	Non-authoritative Information	416	Requested Range Not Satisfiable
204	No Content	417	Expectation Failed
205	Reset Content	418	I'm a teapot
206	Partial Content	421	Misdirected Request
207	Multi-Status	422	Unprocessable Entity
208	Already Reported	423	Locked
226	IM Used	424	Failed Dependency
3XX Redirection		426	Upgrade Required
300	Multiple Choices	428	Precondition Required
301	Moved Permanently	429	Too Many Requests
302	Found	431	Request Header Fields Too Large
303	See Other	444	Connection Closed Without Response
304	Not Modified	451	Unavailable For Legal Reasons
305	Use Proxy	499	Client Closed Request
307	Temporary Redirect	5XX Server Error	
308	Permanent Redirect	500	Internal Server Error
4XX Client Error		501	Not Implemented
400	Bad Request	502	Bad Gateway
401	Unauthorized	503	Service Unavailable
402	Payment Required	504	Gateway Timeout
403	Forbidden	505	HTTP Version Not Supported
404	Not Found	506	Variant Also Negotiates
405	Method Not Allowed	507	Insufficient Storage
406	Not Acceptable	508	Loop Detected
407	Proxy Authentication Required	510	Not Extended
408	Request Timeout	511	Network Authentication Required
		599	Network Connect Timeout Error
H T T P S T A T U S C O D E S			
When a browser requests a service from a web server, an error may occur. This is a list of HTTP status messages that might be returned.			

Frequent Response code

STATUS CODE	EXPLANATION
200 - OK	The request succeeded.
204 - No Content	The document contains no data.
301 - Moved Permanently	The resource has permanently moved to a different URL.
401 - Not Authorized	The request needs user authentication.
403 - Forbidden	The server has refused to fulfill the request.
404 - Not Found	The requested resource does not exist on the server.
408 - Request Timeout	The client failed to send a request in the time allowed by the server.
500 - Server Error	Due to a malfunctioning script, server configuration error or similar.

Difference between 400 and 500 error:

A 4xx code indicates an **error** caused by the user, whereas 5xx codes tell the client that they did everything correctly and it's the server itself who caused the problem.

Difference between 401 and 403 error :

401 Unauthorized response should be used for missing or bad authentication, and a 403 Forbidden response should be used afterwards, when the user is authenticated but isn't authorized to perform the requested operation on the given resource

404 indicates the resource isn't found?

404 error indicates that the server itself was found, but that the server was not able to retrieve the requested page OR your web browser can connect with the server, but the specific page you're trying to access can't be reached.

The easiest way to fix your 404 error code is to redirect the page to another one. You can perform this task using a 301 redirect. What's 301, you may ask? It's a redirect response code that signals a browser that the content has been transferred to another URL.

A 504 Gateway Timeout Error:

It **indicates** that a web server attempting to load a page for you did not get a timely response from another server from which it requested information.

How to Fix the 504 Gateway Timeout Error

1. Retry the web page by selecting the refresh/reload button, pressing F5, or trying the URL from the address bar again. ...
2. Restart all of your network devices. ...
3. Check the proxy server settings in your browser or application and **make** sure they're correct.

405 Error:

Method Not Allowed **response status code** indicates that the request method is known by the server but is not supported by the target resource.

HTTP:

HTTP is a **protocol** which allows the fetching of resources, such as HTML documents.

It's the foundation of any data exchange on the Web and it is a client-server **protocol**, which means requests are initiated by the recipient, usually the Web browser and sent to the server and we receive the response from the server.

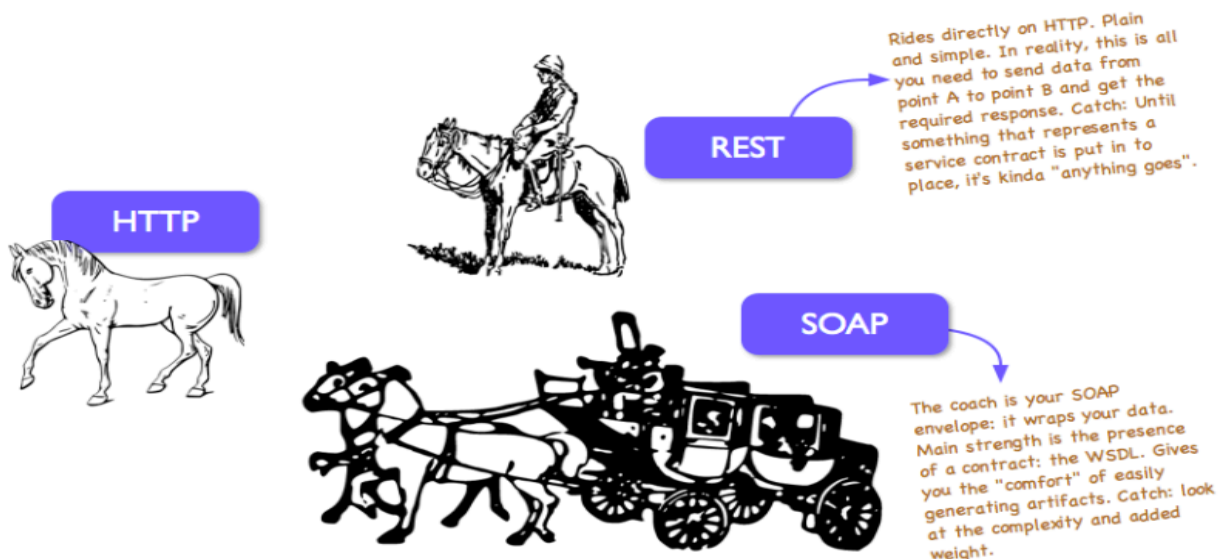


REST, or REpresentational State Transfer, is an architectural style for providing standards between computer systems on the web, making it easier for systems to communicate with each other.

SOAP is an acronym for Simple Object Access Protocol. It is an XML-based messaging protocol for exchanging information among computers.

Difference between rest and soap : ?

REST is almost always going to be faster. The main advantage of SOAP is that it provides a mechanism for services to describe themselves to clients, and to advertise their existence. REST is much more lightweight and can be implemented using almost any tool, leading to lower bandwidth and shorter learning curve. However, the clients have to know what to send and what to expect.

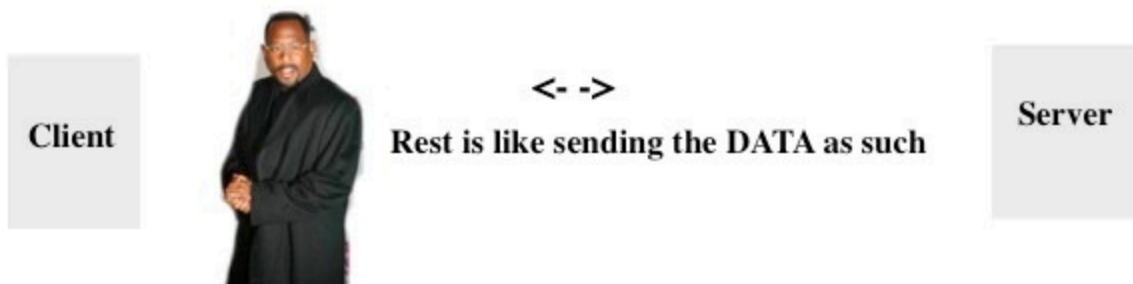


Consider "Martin Lawrence" as your data

SOAP



REST



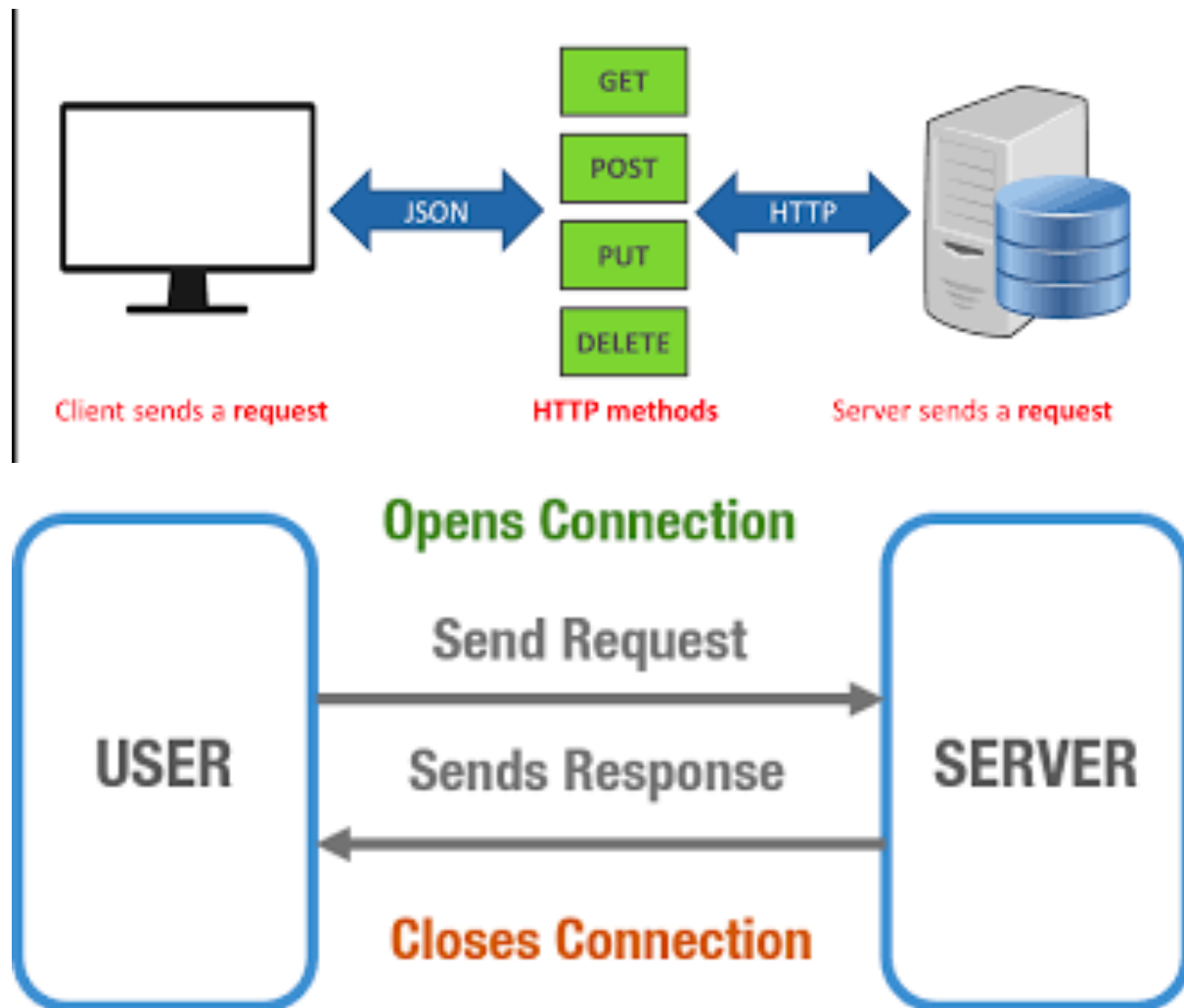
	SOAP	REST
Acronyme	Simple Object Access Protocol	REpresentational State Transfer
Architecture	SOA based architecture that use middleware interoperability.	Architecture style protocol designed for web based communication assuming only point to point communication.
Message Format	XML inside a SOAP Envelope	XML
In business logic	Uses services interfaces to expose the business logic.	Uses URI to expose business logic.
Standards	Defines standards to be strictly followed.	Does not define too much standards like SOAP.
Communication	Use WSDL for communication between consumer and provider.	Uses XML or JSON to send and receive data.
Transport	Transfer is over HTTP, Also uses other protocols such as SMTP, FTP, MIME, JMS, HHTP, etc.	Transfer is over HTTP only.
JavaScript calls	JavaScript can call SOPA but it is difficult to implement.	Easy to call from Javascript.
Security	SOAP defines its own security.	RESTful web services inherits security measures from the underlying transport.
Payload	Payload must support SOAP schema.	Payload can be of any format.
Error Handling	Doesn't support error handling.	Has built-in error handling.
External Calls	Invokes services by calling RPC method.	Simply calls services via URL path.

What is RestAssured?

REST Assured is the best open source tool for Automation Testing. REST Assured is a Java library for validation of REST web services.

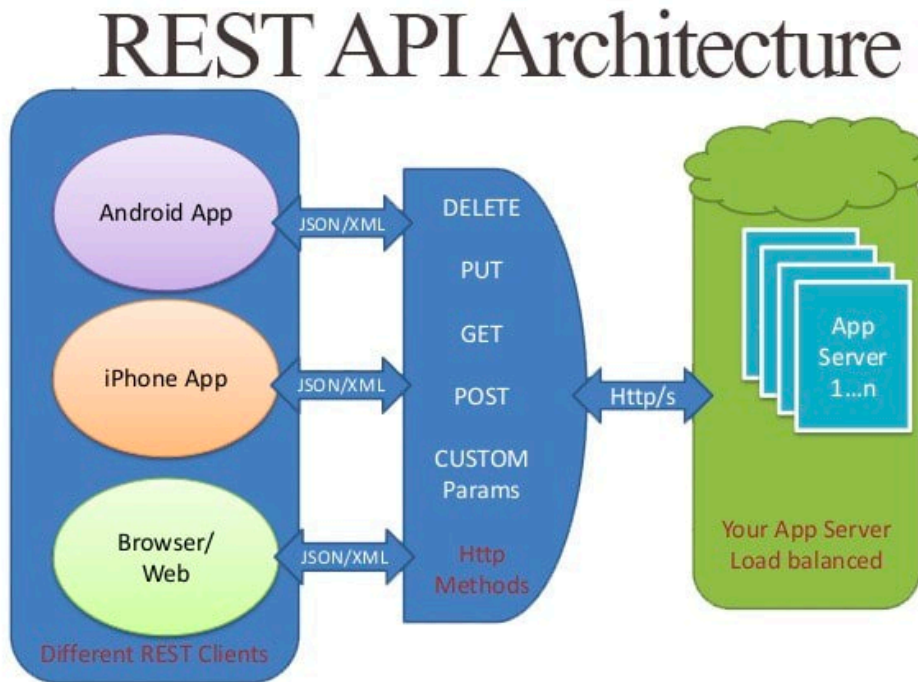
A REST API defines a set of operations where developers can perform requests and receive responses via HTTP protocol.

By **stateless** it means that the server does not store any state about the client session on the server side. The client session is stored on the client. The server is **stateless** means that every server can service any client at any time, there is no session affinity or sticky sessions



What is RestApi? How does it work?

REST stands for Representational state transfer ,
Because REST API's use HTTP, they can be used by practically any programming language.
It acts as medium to propagate communication between the client and server applications on the World Wide Web



Stateless – No client data is stored on the server between requests and session state is stored on the client. http is the transport protocol for REST.

Representational state transfer-- Transferring the State and Representation of a System/Server/Object from one system to another.

Example:(LayMan's Term)

Indigo(Flight Ticket- some representation will be there like how many tickets, graph representation of seat etc) (2. State- Price/Seat of the tikit/Flight will keep changing) .

Here Using RestAPI- instant representation and state of an object we can check.

Getting Started with RestAPI Automation:

Reff: <https://rest-assured.io> Official Website

RestAssured Setup:

REST Assured is a Java API/Library or **expressive programming language(Custom designed to perform specific tasks)** for simplifying testing of REST based services built on top of HTTP Builder. It supports POST, GET, PUT, DELETE, OPTIONS requests and can be used to validate and verify the response of these requests.

Go through the above website & Explore more.

Refer Below:

Step 1) Install Java. Refer to this [link](#) As RestAssured indirectly hit Java API only so we need JAVA

Step 2) Download an IDE/IntelliJ

<https://www.jetbrains.com/idea/download/#section=mac> Select Community Edition

For Eclipse <https://www.eclipse.org/downloads/>

Step 3) InstallMaven and set up your eclipse

Prerequisite: Java & Maven Setup should be done

Rest Assured is Java-based, and knowledge of core Java suffices for learning it

Setup & installation:

- By Downloading jars
- By Using Maven Dependencies

Once setup is done - we are good to Go--->

Basic code:

Reff: <https://github.com/rest-assured/rest-assured/wiki/Usage> Official Website

Note:We can Download Rest Assured Jars from below if not using Maven

<https://github.com/rest-assured/rest-assured/wiki/Downloads>

Add the below dependency to your POM.xml:

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>json-path</artifactId>
  <version>4.3.3</version>
  <scope>test</scope>
</dependency>
```

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>xml-path</artifactId>
  <version>4.3.3</version>
```

```
<scope>test</scope>
</dependency>

<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>json-schema-validator</artifactId>
  <version>4.3.3</version>
  <scope>test</scope>
</dependency>
```

Make sure RestAssured library is Available after setup:

To check use below:

1. Check all the Jars in Project Library
2. Maven clean followed by a maven install, and it should build without any errors.
3. You can add the below lines in your java class and see no compile errors are present.

```
import io.restassured.RestAssured.*;
import io.restassured.matcher.RestAssuredMatchers.*;
import org.hamcrest.Matchers.*;
```

Note*– Now add TestNg/ (JUnit Default) Dependency for Creating Test:

First Sample Rest Assured Code:

The syntax of Rest Assured.io follows BDD style and it's very easily understandable.

Example:

```
Given().  
    baseUrl("www.google.com").  
    param("user", "password").  
    header("place", "Bangalore,india").  
when().get/post/put/delete(endpoint)  
Then().  
    statusCode(XXX).  
    body("all the details fetched");
```

Code	Explanation
Given()	'Given' keyword, Will help in setting the background whatever given or known to us, here, you pass the request headers, query and path param, body, cookies. This is optional if these items are not needed in the request I.e. Given something
When()	'when' keyword defines the Action of your scenario. For example, 'when' you get/post/put/delete something. I.e. when do some action
Then()	Your assert and matcher conditions go here I.e When action is performed then what will happen, we do verification here

Basic get API Automation Test on Dummy API:

Already Tried Dummy API Manual Testing with PostMan, cURL is given below for all:

```
curl -X GET \  
  http://dummy.restapiexample.com/api/v1/employee/2 \  
  -H 'cache-control: no-cache' \  
  -H 'postman-token: 0bab3c22-a90c-2a4d-ac24-5bd103012b0c'
```

```
curl -X POST \  
  http://dummy.restapiexample.com/api/v1/create \  
  -H 'cache-control: no-cache' \  
  -H 'content-type: application/json' \  
  -H 'postman-token: 14b49e79-69f3-0d95-1ed8-39a7c616a8e8' \  
  -d '{  
    "name":"test",  
    "salary":"123",  
    "age":"23"  
  }'
```

```
curl -X PUT \  
  https://reqres.in/api/users/2 \  
  -H 'cache-control: no-cache' \  
  -H 'content-type: application/json' \  
  -H 'postman-token: ae559197-2ac8-6d47-eab8-42d066ee0859' \  
  -d '{  
    "name": "Pankaj",  
    "job": "zion resident"  
  }'
```

```
curl -X DELETE \  
  http://dummy.restapiexample.com/api/v1/delete/2 \  
  -H 'cache-control: no-cache' \  
  -H 'postman-token: bd8028af-1a7a-7ad4-61cf-2e326acaacef'
```

Automating Above API's:

```
package api.testing;  
import io.restassured.RestAssured;  
import io.restassured.RestAssured.*;  
import io.restassured.matcher.RestAssuredMatchers.*;  
import io.restassured.response.Response;  
import org.hamcrest.Matchers.*;  
import org.junit.Assert;  
import org.junit.Test;
```

```
import static io.restassured.RestAssured.given;

public class GetEmployeeTest {
    //GET Call Using RestAssured Automation
    @Test
    public void getEmployee()
    {
        //Passing Base URL
        RestAssured.baseURI="http://dummy.restapiexample.com/api/v1";
        Response res=given()
            .when().get("/employee/2")
            .then().extract().response();
        int Statuscode=res.getStatusCode();
        System.out.println("Status code of the API is = "+Statuscode);
        Assert.assertEquals(Statuscode, 200);//verifying the status code
    }
    //POST Call Using RestAssured Automation
    @Test
    public void createEmployee()
    {
        //Passing Base URL
        String body="{\n" +
            "\t\"name\": \"test\", \n" +
            "\t\"salary\": \"123\", \n" +
            "\t\"age\": \"23\" \n" +
            "}";
        RestAssured.baseURI="http://dummy.restapiexample.com/api/v1";
        Response res=given()
            .header("Content-Type", "application/json")
            .body(body)
            .when().post("/create")
            .then().extract().response();
        int Statuscode=res.getStatusCode();
        System.out.println("Status code of the API is = "+Statuscode);
        Assert.assertEquals(Statuscode, 200);//verifying the status code
    }
    //PUT Call Using RestAssured Automation
    @Test
    public void updateEmployee()
    {
        //Passing Base URL
        String body="{\n" +
            "  \"name\": \"Pankaj\", \n" +
            "  \"job\": \"zion resident\" \n" +
            "}";
        RestAssured.baseURI="https://reqres.in/api/";
        Response res=given()
            .header("Content-Type", "application/json")
            .body(body)
            .when().put("/users/2")
            .then().extract().response();
        int STATUSCODE=res.getStatusCode();
        System.out.println("Status code of the API is = "+STATUSCODE);
        Assert.assertEquals(STATUSCODE, 200);//verifying the status code
    }
}
```

```
}  
//DELETE Call Using RestAssured Automation  
@Test  
public void deleteEmployee()  
{  
    //Passing Base URL  
    RestAssured.baseURI="http://dummy.restapiexample.com/api/v1";  
    Response res=given()  
        .when().delete("/delete/2")  
        .then().extract().response();  
    int STATUSCODE=res.getStatusCode();  
    System.out.println("Status code of the API is = "+STATUSCODE);  
    Assert.assertEquals(STATUSCODE, 200);//verifying the status code  
}  
}
```

So Basically we are using restAssured Basic Syntax like below:

BaseURL

given(): (What and all the resources we have)

Request header, Parameter, Request Cookies, body etc

when(): (When we perform some Action)

get(resource) , post(resource), put(resource), Delete(resource) etc.

then(): (Do Validation/Assertion to make sure we are getting expected output)

Assert.assertEquals(Actual, Expected);

extract(): (Extract the data from Response body)

Extract the data from the Response body for future/Further use.

Now Let's Work on LIVE APPLICATION :

We can try for any of given below (Will see for any one)

1. Google Map Example
2. Whether Example
3. Twitter real time Example

We will take Twitter Web Services example for Automation:

Twitter uses **OAuth** to Provide authorize Access to his API

<https://developer.twitter.com/en/docs/basics/authentication/overview/oauth>

There are two ways to Authenticate:

Application-only authentication

Application-user authentication

Go to the official website of twitter for API Docs- Just Search Rest API for Twitter

Prerequisite:

1. Twitter Account
2. You need a website from which you want to connect to twitter. (will give dummy Website)

Just go to the URL below to get OAuth:

<https://developer.twitter.com/>

Go to Apps

<https://developer.twitter.com/en/apps>

There are three Token we need

Go to App and Create an App.

For that we need to Registered as Developer Account

Then fill all the details and complete the Email verification then only you will be able to create APP. (Give any website URL and Some Dummy Name)

Create Access Token & token Secret (We can Regenerate as well- For Security Purpose)

wgSOcQ40V4Pe40a2fePq5p0i6n (API key)

VdBMjQxoqnyofBRwkyI2MObC7y8UNFK0qXBWSfo4BjMVPDIkDFr (API secret key)

764067006-DfszmyVtxVKOT8hVT8h9uZ1z6QFz0qRCNLxpEQkrd (Access token)

b9nD2H0zzGp73Nw4zellpDeTVKeJbwqedq93TZ0074qHI1 (Access token secret)

Now Go to Twitter Developer section :

<https://developer.twitter.com/en/docs/tweets/timelines/overview>

Now will use `GET statuses/home_timeline`

What and all we need to pass as request we need to explore here:

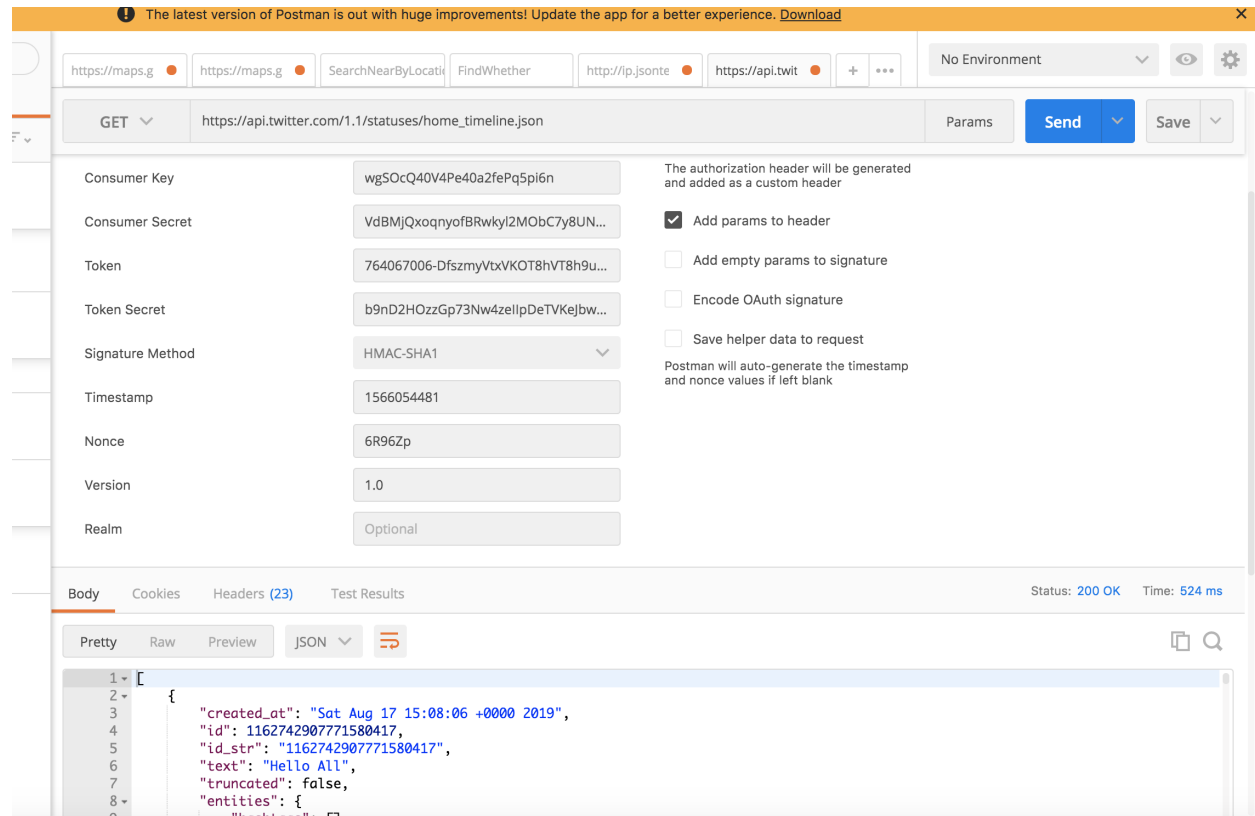
Go to PostMan give URL and Method As per documentation of Twitter:

Select `get()` and pass- https://api.twitter.com/1.1/statuses/home_timeline.json

Then Add OAuth Tokens and Hit the URL we will get All the Twits whatever made by Me at my times line

Post Twitte:<https://developer.twitter.com/en/docs/twitter-api/v1/tweets/post-and-engage/overview>

See the output below:



Post Tweets on timeline:

```
curl -X POST \
  https://api.twitter.com/1.1/statuses/update.json \
  -H 'authorization: OAuth
oauth_consumer_key=\"wgSOcQ40V4Pe40a2fePq5pi6n\",oauth_token=\"764067006-
dTTC3u4MMcCbdvE6BGILpYknLsbmzKvaoOZnkcZ\",oauth_signature_method=\"HM
AC-
SHA1\",oauth_timestamp=\"1609076253\",oauth_nonce=\"bep5l9\",oauth_version=\"1.0
\",oauth_signature=\"6DjLf7U9l5B1pPdU63m36FuZ8gU%3D\"' \
  -H 'cache-control: no-cache' \
  -H 'content-type: multipart/form-data; boundary=----
WebKitFormBoundary7MA4YWxkTrZu0gW' \
  -H 'postman-token: 94fb97a0-ce59-9b7f-a962-66731b5bd34f' \
  -F 'status=Hello All'
```

GET Tweet from timeline:

```
curl -X GET \
  https://api.twitter.com/1.1/statuses/home_timeline.json \
  -H 'authorization: OAuth
oauth_consumer_key=\"wgSOcQ40V4Pe40a2fePq5pi6n\",oauth_token=\"764067006-
dTTC3u4MMcCbdvE6BGILpYknLsbmzKvaoOZnkcZ\",oauth_signature_method=\"HM
AC-
SHA1\",oauth_timestamp=\"1609076205\",oauth_nonce=\"Suie7l\",oauth_version=\"1.0
\",oauth_signature=\"A9dUzpQnAeM%2FqvAfrwNor2Ne9l%3D\"' \
  -H 'cache-control: no-cache' \
  -H 'postman-token: fc529b16-60b9-5e4a-77a9-e929dd1931a4'
```

Delete Tweet:

```
curl -X POST \
  https://api.twitter.com/1.1/statuses/destroy/1343189310976081921.json \
  -H 'authorization: OAuth
oauth_consumer_key=\"wgSOcQ40V4Pe40a2fePq5pi6n\",oauth_token=\"764067006-
dTTC3u4MMcCbdvE6BGILpYknLsbmzKvaoOZnkcZ\",oauth_signature_method=\"HM
AC-
SHA1\",oauth_timestamp=\"1609076271\",oauth_nonce=\"fRcZ9w\",oauth_version=\"1.
0\",oauth_signature=\"AuEShG3PrCOSS8C1%2FH7qPoGtZT4%3D\"' \
  -H 'cache-control: no-cache' \
```

```
-H 'postman-token: 3f246a34-977b-9011-ef15-a835ae8fe59c'
```

If we have multiple Tweets we can make the count as a parameter and use in our API call, lets see

Add Parameter count=1 in API we will get only one updated tweet.

That's how the get() method works.

Let's see by Using our Automation- by RestAssured.

Will just refer to Postman and we can write our Scripts.

We need below jars to use Twitter API:

```
<!-- https://mvnrepository.com/artifact/com.github.scribejava/scribejava-apis -->
```

```
<dependency>
```

```
  <groupId>com.github.scribejava</groupId>
```

```
  <artifactId>scribejava-apis</artifactId>
```

```
  <version>2.5.3</version>
```

```
</dependency>
```

Example:

```
package RestAssuredTest;
import org.testng.Assert;
import org.testng.annotations.Test;
import io.restassured.RestAssured;
import io.restassured.path.json.JsonPath;
import io.restassured.response.Response;
import static io.restassured.RestAssured.given;
import java.util.ArrayList;

public class TweeterTest_01
{
    String consumerKey="wgSOcQ40V4Pe40a2fePq5pi6n";
    String consumerSecrete="VdBMjQxoqnyofBRwkyI2MOBc7y8UNFKqXBWSfo4BjMVPDIkDFr";
    String Tokens="764067006-DfszmyVtxVKOT8hVT8h9uZ1z6QFzqRCNLxpEQkrd";
    String TokenSecret="b9nD2HOzzGp73Nw4zellpDeTVKeJbwqedq93TZ074qHl1";
    @Test
    public void getTweetTest()
    {
        //Passing Base URL
        RestAssured.baseURI="https://api.twitter.com/1.1/statuses";
        Response res=given().auth().oauth(consumerKey, consumerSecrete, Tokens,
TokenSecret)
```

```

        .queryParams("count", 1)
        .when().get("/home_timeline.json")
        .then().extract().response();
        //Converting Response into String Object
        String response=res.asString();
        System.out.println(response);//All the response will be there
        //Now we want to iterate the Response and since response is in Json so lets use
        JsonPath

        JsonPath j=new JsonPath(response);
        //now by j ref call the method to get the text
        ArrayList<String> t=j.get("text");//Hold in string- Refer the postman for Json Key
        System.out.println("Text available into Tweeter is "+t);
    }
    @Test
    public void getTweetTestValidate()
    {
        //Passing Base URL
        RestAssured.baseURI="https://api.twitter.com/1.1/statuses";
        Response res=given().auth().oauth(consumerKey, consumerSecret, Tokens,
        TokenSecret)

        .queryParams("count", 1)
        .when().get("/home_timeline.json")
        .then().extract().response();
        int Statuscode=res.getStatusCode();
        Assert.assertEquals(Statuscode, 200);//verifying the status code
    }
    @Test
    public void postTweet()
    {
        //Passing Base URL
        RestAssured.baseURI="https://api.twitter.com/1.1/statuses";
        Response res=given().auth().oauth(consumerKey, consumerSecret, Tokens,
        TokenSecret)

        .queryParams("count", 1)
        .when().post("/home_timeline.json")
        .then().extract().response();
        int Statuscode=res.getStatusCode();
        Assert.assertEquals(Statuscode, 200);//verifying the status code
    }
}

```

Refer this for more details:

<https://docs.google.com/document/d/17dMI7ZeUQb2anGBxf2xhE5XsdXE2MB12sri7iaOqsAM/edit>

Post() Method for post a Tweet:

If its get request we can use Query parameter, But if its post we have to use form-Parameter in body:

Post Tweet Using PostMan-

Now do the same suing Automation - RestAssured

```
package RestAssuredTest;
import org.testng.Assert;
import org.testng.annotations.Test;
import io.restassured.RestAssured;
import io.restassured.path.json.JsonPath;
import io.restassured.response.Response;
import static io.restassured.RestAssured.given;
import java.util.ArrayList;

public class TweeterTest_01
{
    String consumerKey="wgSOcQ40V4Pe40a2fePq5pi6n";
    String consumerSecrete="VdBMjQxoqnyofBRwkyl2MObC7y8UNFKqXBWSfo4BjMVPDIkDFr";
    String Tokens="764067006-DfszmyVtxVKOT8hVT8h9uZ1z6QFzqRCNLxpEQkrd";
    String TokenSecret="b9nD2HOzzGp73Nw4zellpDeTVKeJbwqedq93TZ074qHl1";
    // @Test
    public void getTweetTest()
    {
        // Passing Base URL
        RestAssured.baseURI="https://api.twitter.com/1.1/statuses";
        Response res=given().auth().oauth(consumerKey, consumerSecrete, Tokens,
TokenSecret)
        .queryParams("count", 1)
        .when().get("/home_timeline.json")
        .then().extract().response();
        // Converting Response into String Object
        String response=res.asString();
        System.out.println(response); // All the response will be there
        // Now we want to iterate the Response and since response is in Json so lets use
JsonPath
        JsonPath j=new JsonPath(response);
        // now by j ref call the method to get the text
        ArrayList<String> t=j.get("text"); // Hold in string- Refer the postman for Json Key
        System.out.println("Text available into Tweeter is "+t);
        System.out.println("Id available into Tweeter is "+j.get("id"));

    }
    // @Test
    public void getTweetTestValidate()
    {

```

```

//Passing Base URL
RestAssured.baseURI="https://api.twitter.com/1.1/statuses";
Response res=given().auth().oauth(consumerKey, consumerSecret, Tokens,
TokenSecret)
    .queryParam("count", 1)
    .when().get("/home_timeline.json")
    .then().extract().response();
int Statuscode=res.getStatusCode();
Assert.assertEquals(Statuscode, 200);//verifying the status code
}
//@Test
public void postTweet()
{
    //Passing Base URL
    RestAssured.baseURI="https://api.twitter.com/1.1/statuses";
    Response res=given().auth().oauth(consumerKey, consumerSecret, Tokens,
TokenSecret)
        .queryParam("status", "I am creating Tweets via API")
        .when().post("/update.json")
        .then().extract().response();
        int Statuscode=res.getStatusCode();
        Assert.assertEquals(Statuscode, 200);//verifying the status code
    }
    //Delete the Tweets
    @Test
    public void deleteTweet()
    {
        //Passing Base URL
        RestAssured.baseURI="https://api.twitter.com/1.1/statuses";
        Response res=given().auth().oauth(consumerKey, consumerSecret, Tokens,
TokenSecret)
            .when().post("/destroy/1163040708581715969.json");//But this should not be the
HardCoded
            .then().extract().response();
            int Statuscode=res.getStatusCode();
            Assert.assertEquals(Statuscode, 200);//verifying the status code
        }
    }
}

```

Now in One Test we will see how to create a post tweet then verify it by Getting it and then delete it--end to end.

<https://jsonformatter.org/> -

That's All.

Hello Team,

Please give your Valuable Review/Rating,

I need your help to keep myself Motivated to create exciting stuff for you.

Your Rating is most valuable for me.

Please find out more course's by me with **Discount Coupon:**

JAVA-SELENIUM:

<https://www.udemy.com/course/selenium-webdriver-with-java-testng-maven-git-jenkins/>

XPATH & CSS:

<https://www.udemy.com/course/learn-element-locators-css-selector-and-xpath-from-scratch/>

GIT & GITHUB Basics:

<https://www.udemy.com/course/learn-git-from-basic/>

TestNG

<https://www.udemy.com/course/testng-framework/>

JAVA Basics:

<https://www.udemy.com/course/java-for-beginner/>

Please email me your current course and the **Discount Coupon code you need for the course** at given address: **pankajonlinetutor@gmail.com**

Learning Tips:

1. Please Practice all the Example yourself whatever I have covered.
2. Everything looks very easy while watching, Once you go through then you will be more cleared about the topic.

I need your support, **Please help me with your Ratings and Review.**

Thank you !

Keep Learning :: Keep Growing